

Raspodijeljeni sustavi, Predavanje 01

Osnove i modeli sustava

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

1 Ishodi učenja

Nakon ovog predavanja trebali biste moći:

1. Prepoznati procese, poruke i zajednički cilj u raspodijeljenoj aplikaciji.
2. Objasniti jedan razlog za raspodjelu sustava i trošak koordinacije koji ona donosi.
3. Razlikovati konkurentnost od paralelizma na vremenskoj crti izvođenja.
4. Objasniti djelomični ispad i zašto istek vremena čekanja (timeout) ne utvrđuje ishod udaljene operacije.
5. Navesti osnovne pretpostavke o mreži, čvorovima i vremenu te prepoznati pravilo koje sustav uvijek mora poštovati.[1]

Predznanje: osnove programiranja, funkcije, procesi i neformalno razumijevanje mreže. Ne pretpostavlja se prethodno znanje raspodijeljenih algoritama.

Opseg predavanja: ove osnove podupiru kasnija predavanja o arhitekturama, RPC-u, satovima, replikaciji i konsenzusu. Uvodimo pitanja na koja ti mehanizmi odgovaraju, ali ih još ne implementiramo. Primjeri opisuju zamišljeni sveučilišni sustav za predaju projekata, a ne stvarnu infrastrukturu sveučilišta.

[1] Takvo se pravilo naziva **invarijanta**. Na primjer, ponovni pokušaj iste predaje projekta nikada ne smije stvoriti drugi prihvaćeni zapis. Prisjetite se **idempotentnosti** iz Web aplikacija: ponavljanje operacije ima isti učinak kao da je izvedena jednom. Ako su ponovni pokušaji predaje idempotentni, lakše je očuvati ovu invarijantu; odgovor na svaki pokušaj pritom ne mora biti jednak.

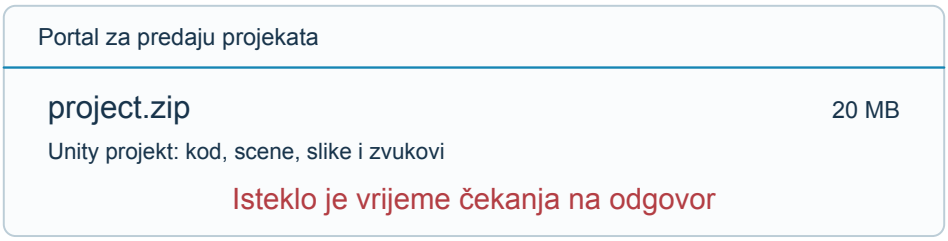
2 Primjer: je li moja predaja uspjela?

Studentica je prijavljena na sveučilišni portal za predaju projekata i već je upisana na kolegij. Za zadatak je izradila malu igru u Unityju. Sve datoteke projekta zapakirala je u jednu **ZIP datoteku**: izvorni kod, scene i resurse poput slika i zvukova. Ta **arhiva projekta od 20 MB** datoteka je koju želi predati nastavniku. Odabere je u portalu i pritisne **Predaj**. Nakon čekanja preglednik prikaže istek vremena čekanja. Je li portal prihvatio projekt za nastavnika? Treba li ga predati ponovno?

Prijenos bajtova i prihvaćanje predaje različiti su koraci. Čak ni traka napretka na 100 % ne dokazuje da je portal provjerio pravila, spremio cijelu datoteku i zabilježio prihvaćenu predaju. U našem primjeru uspjeh znači da su cijeli projekt i zapis o njegovoj predaji pohranjeni i dostupni nastavniku za preuzimanje; ne znači da ga je nastavnik otvorio ili ocijenio.

Mogućih je objašnjenja nekoliko. Cijeli zahtjev za predaju možda nikada nije stigao do usluge; prekinut prijenos može ostaviti samo privremene bajtove. Usluga ga je možda primila, ali stala prije nego što je išta pohranila. Možda je pohranila predaju, a zatim izgubila odgovor. Ili zahtjev još čeka u redu. Isti vidljivi simptom može, dakle, odgovarati različitim unutarnjim stanjima.

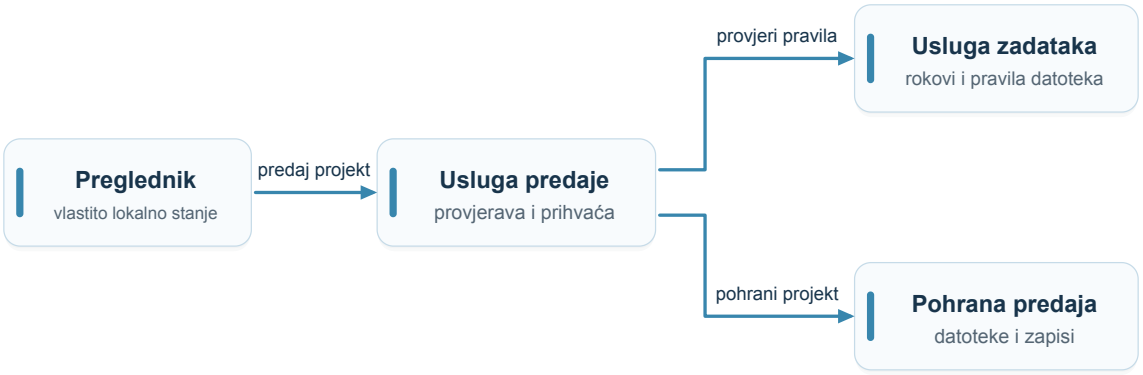
To je središnji problem ovog predavanja: **proces mora donositi odluke na temelju nepotpunih informacija o drugim procesima**. Pitanju ćemo se vratiti nakon što izgradimo model sustava. Primjer razrađuje problem neizvjesnog ishoda opisan u [3, odjeljak 2.1].



Slika 1: Zamišljeni portal javlja istek vremena čekanja bez potvrde o prihvaćanju.

2.1 Granica sustava

Preglednik studentice šalje projekt **usluzi predaje**. **Usluga zadataka** daje podatke o zadatku, rokove te dopuštene vrste i veličine datoteka. Usluga predaje provjerava ta pravila i sprema cijeli projekt i zapis o prihvaćenoj predaji u **pohranu predaja**. Pretpostavljamo da je studentica već prijavljena i upisana, pa provjere autentičnosti i upisa nisu dio ovog uvodnog dijagrama. Ova četiri sudionika koji surađuju čine granicu sustava u primjeru.



Strelica: pozivatelj → potrebna komponenta. Oznaka: zahtjev. Odgovori su izostavljeni.
Okviri su logički procesi, ne zadani broj računala.

Slika 2: Primjer predaje sadrži zasebne procese povezane zahtjevima. Svaki okvir ima vlastito lokalno stanje. Strelica pokazuje od pozivatelja prema komponenti o kojoj za ovu operaciju ovisi; oznaka imenuje zahtjev. Odgovori su izostavljeni.

Proces je program koji se izvršava. **Čvor** (node) je jedinica koju u određenom modelu smatramo neovisnim sudionikom: može biti proces ili računalo. Moramo reći na koje značenje mislimo. Više procesa može dijeliti isto fizičko računalo, pa dijagram s četiri procesa ne znači četiri zasebna računala ni četiri neovisne domene hardverskih ispada.

Svaku strelicu čitajte kao **„poziva i ovisi o”**: preglednik poziva uslugu predaje, a usluzi predaje trebaju usluga zadataka za provjeru pravila i pohrana predaja za spremanje projekta. Strelica pokazuje prema komponenti koja se poziva; ne znači da informacije teku samo u jednom smjeru. Odgovori se vraćaju, ali su u ovom pregledu izostavljeni. To su ovisnosti za prikazanu operaciju predaje, a ne karta svih mogućih ovisnosti u sustavu.

Usluga predaje ne može jednostavno pročitati varijablu u udaljenom procesu usluge zadataka kao da je lokalna varijabla. U našem modelu ona traži informaciju slanjem poruke. Čak i kada API skriva tu razmjenu, informacija i dalje mora prijeći komunikacijsku granicu.

3 Što sustav čini raspodijeljenim?

U ovom predavanju koristimo sljedeću definiciju.

Definicija 1.1

Raspodijeljeni sustav (distributed system) čine komponente koje se neovisno izvršavaju te komuniciraju i koordiniraju razmjenom poruka radi zajedničkog cilja.

Aplikacija može nastojati korisnicima pružiti jedinstvenu, cjelovitu uslugu, iako se posao obavlja na više mjesta. Time odvajamo strukturu sustava od korisnog cilja dizajna: skrivanja nepotrebnih pojedinosti raspodjele [3, odjeljak 1.1].

Tri pitanja pomažu primijeniti definiciju:

- **Tko djeluje neovisno?** Prepoznajte procese koji sudjeluju i njihovo lokalno stanje (local state).
- **Kako komuniciraju?** Prepoznajte zahtjeve, odgovore, događaje i granice koje prelaze.
- **Što zajedno žele postići?** Navedite operaciju vidljivu korisniku ili zajednički cilj.

Mapa s nekoliko nepovezanih programa nije dovoljna da bi se pokazala raspodijeljena aplikacija. S druge strane, jedno ime proizvoda ili jedna web-adresa ne znače da sudjeluje samo jedan proces.

3.1 Primjeri i protuprimjeri

Preglednik, API i baza koji se izvršavaju kao zasebni procesi na istom računalu koordiniraju razmjenom poruka kako bi obradili zahtjeve. Zadovoljavaju našu definiciju utemeljenu na komponentama, ali dijele domenu ispada istog računala. Računala kojima su dodijeljeni različiti kadrovi istog filma također koordiniraju prema zajedničkom rezultatu i zadovoljavaju našu definiciju. Tri nepovezana prijenosna računala na istoj Wi-Fi mreži ne zadovoljavaju je: sam pristup mreži ne daje ni koordinaciju ni zajednički zadatak. Dretve koje komuniciraju samo preko zajedničke memorije ilustriraju konkurentnost, ali ne zadovoljavaju kriterij razmjene poruka iz ove definicije.

API i radni proces (worker) u zasebnim kontejnerima na jednom prijenosnom računalu zadovoljavaju našu definiciju kada koordiniraju porukama. I dalje dijele domenu ispada računala; takav razmještaj ne podnosi gubitak tog računala. Neke definicije pojam „raspodijeljeni sustav” rezerviraju za više računala, pa navedite definiciju koju koristite.

Konkretan primjer razmjene poruka: **tri MPI procesa računaju prosječnu temperaturu u velikom skupu podataka**. Proces 0 dijeli mjerenja u tri dijela, jedan zadržava, a ostale šalje procesima 1 i 2. Svaki proces računa lokalni zbroj i broj mjerenja. Radni procesi šalju rezultate procesu 0, koji ukupni zbroj dijeli ukupnim brojem mjerenja. Procesi djeluju neovisno i koordiniraju porukama prema zajedničkom cilju. Mogu raditi na jednom ili više računala; to mijenja domene ispada, ali ne i strukturu razmjene poruka u ovom primjeru.

3.2 Cjelovitost i transparentnost

Sustav za predaju treba pružati smislenu uslugu: studenti odaberu zadatak, prenesu projekt i dobiju razumljiv rezultat. Ne bi trebali birati računalo s bazom ni razumjeti kako je usluga zadataka razmještena. **Transparentnost raspodjele** (distribution transparency) znači skrivanje odabranih pojedinosti implementacije od korisnika ili pozivatelja.

Transparentnost nije obećanje da komunikacija ništa ne košta ni da se ispadi uvijek mogu sakriti. Na primjer, zamjena neizvjesnog rezultata samouvjerenom porukom „Predaja nije

uspjela” zavarava ako predaja možda već postoji. Korisno sučelje može umjesto toga prikazati „Ishod nepoznat; provjeravamo stanje” i ponuditi način da se rezultat dohvati. Pravilo prikaza mora odgovarati onome što sustav stvarno zna [2].

Potpunu transparentnost nećemo zahtijevati za svaku aplikaciju. Oznaka da ste izvan mreže ili izričit status sinkronizacije mogu biti korisna informacija, a ne pogreška u dizajnu.

4 Zašto raspodijeliti posao?

Tri razloga objašnjavaju većinu raspodijeljenih sustava. Prvo, **komponente su fizički odvojene**: korisnici, uređaji i izvori podataka nalaze se tamo gdje jesu, a sustav do njih mora doprijeti. Drugo, **jedno računalo nema dovoljno kapaciteta** za posao. Treće, **sustav mora nastaviti raditi kada neki dio ispadne**, a to traži redundantne kopije usluga i podataka. Budući da se gotovo svaki informacijski sustav suočava s barem jednom od tih potreba, gotovo je svaki informacijski sustav raspodijeljen. Čak i jednostavna web-aplikacija ima preglednik, poslužitelj i bazu kao zasebne procese, često na zasebnim računalima.

Razlog	Primjer sustava za predaju	Novo pitanje
Komponente su fizički odvojene	Studenti predaju od kuće, s fakulteta ili iz inozemstva; senzori i mobiteli u drugim sustavima šalju podatke s mjesta na kojima se nalaze.	Koliko poruke putuju i što se događa kada veza pukne?
Potreban je veći kapacitet	Pred rok više radnih procesa obrađuje različite projekte.	Kako izbjeći da dvaput prihvate istu predaju?
Sustav mora preživjeti ispade	Kopije usluge i pohranjenih projekata (redundancija) omogućuju nastavak rada kada jedna stane.	Koje stanje preživljava i kako klijent pronalazi zamjenu?

Dizajn raspodijeljenih sustava ima i daljnje ciljeve; udžbenik [1] o njima govori detaljnije. **Dijeljenje resursa** omogućuje da više kolegija koristi istu pohranu ili ista pravila zadataka. **Transparentnost** studentima daje jedan cjelovit portal, bez potrebe da biraju radni proces. **Otvorenost** znači da komponente slijede dogovorena sučelja pa, na primjer, fakultetska usluga zadataka koju vodi drugi tim i portal za predaju mogu surađivati; to ne znači nužno softver otvorenog koda. To su ciljevi kojima težimo, a ne automatske posljedice dodavanja računala, i nijedan ne uklanja potrebu za razmišljanjem o koordinaciji [1].

Vertikalno skaliranje (scaling up) znači povećanje kapaciteta postojeće komponente. **Horizontalno skaliranje** (scaling out) znači dodavanje komponenti. Horizontalno skaliranje može povećati kapacitet kada se posao može učinkovito podijeliti, ali zajedničko usko grlo

ili dodatna koordinacija mogu ograničiti korist. Dodavanje poslužitelja ne čini operaciju automatski ispravnom ni bržom.

Za malu aplikaciju može biti dovoljan jedan dobro razumljiv proces i pohrana predaja. Raspodjelu opravdava konkretan zahtjev, a ne broj okvira na dijagramu arhitekture. Cijena uključuje komunikaciju, razmještaj, oporavak i razmišljanje o stanjima koja različite komponente mogu vidjeti u različito vrijeme.

4.1 Facebook: veličina kao motivacija

Facebook je objavio **1 milijardu mjesečno aktivnih korisnika u listopadu 2012.**, **2,50 milijardi u prosincu 2019.** i **3,07 milijardi u prosincu 2023.** To su brojevi mjesečne aktivnosti, a ne istodobni korisnici ni zahtjevi u sekundi. Facebook je od prvog tromjesečja 2024. prestao objavljivati zasebne dnevne i mjesečne pokazatelje aktivnosti; noviji podaci za obitelj Metinih proizvoda objedinjuju više proizvoda i nisu nastavak ovog niza. Izvori: prekretnica 2012., rezultati za 2019., rezultati za 2023. i promjena izvještavanja.

Rast broja korisnika i njihove aktivnosti motivira raspodjelu računanja i podataka kada resursi jednog računala više nisu dovoljni. Facebookov inženjerski prikaz skaliranja među tehnikama navodi particioniranje i priručnu pohranu (caching). Ti mehanizmi donose posao koordinacije i oporavka. Ovo predavanje uvodi vertikalno i horizontalno skaliranje; predavanje 11 razrađuje particioniranje, raspodjelu opterećenja, priručnu pohranu i uska grla. Asinkrona komunikacija pripada predavanjima 03 i 10, a replikacija predavanju 06.

5 Moraju li svi čekati jednu predaju?

Zamislite veliko sveučilište na kojemu mnogo studenata predaje projekte neposredno pred rok. Primanje ZIP datoteke samo je prvi korak. Portal provjerava pravila zadatka, raspakirava i skenira datoteke, sprema projekt i bilježi njegovo prihvaćanje. Pretpostavimo da ti koraci traju **20 minuta za jednu predaju**.

Znači li to da nitko drugi ne može predati projekt tijekom tih 20 minuta? Ne nužno. Moramo znati kako je posao organiziran:

- Ako **jedan radni proces obrađuje jedan cijeli posao za drugim**, kasniji poslovi čekaju u redu. Uz fiksnih 20 minuta obrade i neprekidan rad taj radni proces dovrši tri posla na sat. Drugi studenti i dalje mogu prenositi datoteke ako se primanje prijenosa obavlja zasebno.
- Ako se velik dio tih 20 minuta **čeka na pohranu ili drugu uslugu**, radni proces za vrijeme tih čekanja može obrađivati dijelove drugih predaja. Više poslova može biti u tijeku, a da se njihove instrukcije ne izvršavaju istodobno.
- Uz **više radnih procesa i dovoljno resursa** različiti se projekti mogu obrađivati istodobno. Dva neovisna radna procesa u idealnom slučaju mogu dovršiti šest takvih poslova na sat, ali samo ako zajednička pohrana, mreža i ostale provjere ne postanu uska grla.

Vrijeme jedne predaje stoga ne određuje koliko predaja cijeli sustav može dovršiti na sat. **Kako dovršiti više posla u istom vremenu, a da svaki projekt i dalje bude ispravno prihvaćen?** Brzina dovršavanja posla naziva se **propusnost** (throughput). Smanjenje čekanja jednog studenta drugi je cilj. Sljedeći odjeljak imenuje ove različite načine organizacije posla.

6 Konkurentnost i paralelizam

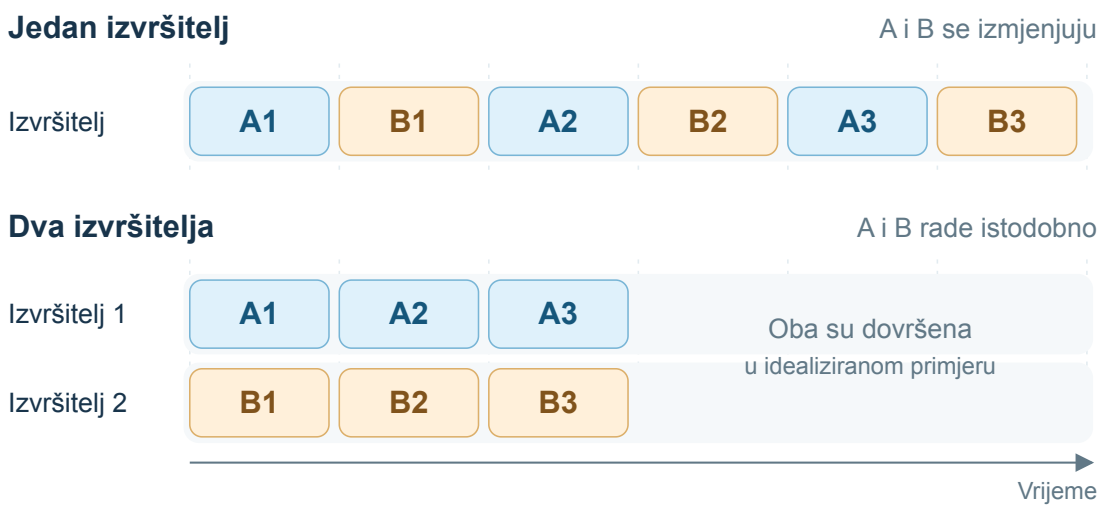
Neka su A i B predaje različitih studenata. Radni proces može napredovati s B dok A čeka pohranu, ili dva radna procesa mogu istodobno obrađivati A i B. Te alternative odgovaraju na pitanje o opterećenju i motiviraju sljedeće pojmove. Njihove se instrukcije ne moraju izvršavati u istom trenutku: jedan radni proces može se prebacivati između njih. S dvije procesorske jezgre oba se zadatka doista mogu izvršavati istodobno [2].

Definicija 1.2

Konkurentnost (concurrency): više je zadataka u tijeku tijekom razdoblja koja se preklapaju.

Definicija 1.3

Paralelizam (parallelism): više se zadataka izvršava istodobno.



Slika 3: Gornja vremenska crta isprepliće zadatke A i B na jednom radnom procesu. Donje crte izvršavaju A i B istodobno na zasebnim radnim procesima. Svaki označeni blok dio je zadatka, ne nužno jedna instrukcija.

Prisjetite se provjera od 20 minuta: to je trajanje značilo tri posla na sat samo uz pretpostavku jednog radnog procesa koji poslove obrađuje strogo jedan za drugim.

Preklapanje posla može zaposliti resurse tijekom čekanja, a paralelno izvršavanje može povećati broj dovršenih poslova na sat ako zajednički resursi to prate. Isti obrazac izvođenja kasnije ćemo ponovno upotrijebiti za razmišljanje o dvama pokušajima iste predaje.

Konkurentnost se može pojaviti unutar jednog računala; ne zahtijeva raspodijeljeni sustav. Raspodjela dodaje komunikaciju između sudionika koji se neovisno izvršavaju. I paralelizam se može pojaviti unutar jednog računala, pa više jezgri samo po sebi ne dokazuje raspodijeljenu aplikaciju.

Prema našoj definiciji paralelni se zadaci također preklapaju, pa je paralelno izvršavanje konkurentno; konkurentni se zadaci ne moraju izvršavati paralelno. Jedan radni proces može se prebaciti pri čekanju, dobrovoljnom prepuštanju (yield) ili kada ga raspoređivač prekine. Ovisnosti i dalje ograničavaju redoslijed koraka. Konkurentnost ne znači da proizvoljan redoslijed čuva rezultat: nesinkronizirano zajedničko stanje može izazvati utrke (races).

6.1 Kada izvršitelj mijenja zadatak?

Zadatak se može promijeniti na tri načina (slika 4). Prvo, zadatak **čeka ulaz/izlaz**, na primjer pohranu, pa izvršitelj u međuvremenu nastavlja drugi zadatak. Drugo, **operacijski sustav prekida dretvu** kada istekne njezin vremenski odsječak; dretva to ne bira i ne zna gdje će stati. Treće, zadatak može **dobrovoljno ustupiti red** i bez ulaza/izlaza.



Slika 4: Tri točke promjene zadatka na jednom izvršitelju. Kooperativno raspoređivanje (asyncio) mijenja zadatak samo na `await`; preemptivno raspoređivanje dretvi operacijskog sustava može ga promijeniti između bilo koje dvije instrukcije.

U `asyncio` jedna dretva izvršava više korutina, a korutina se mijenja samo na `await`. Operacijski sustav i dalje može pauzirati cijeli proces, ali tada ne radi nijedna druga korutina iz iste petlje događaja. Duga petlja koja samo računa zato zauzima petlju događaja; `await asyncio.sleep(0)` ne čeka ništa, nego samo ustupa red drugim zadacima:

```
async def provjeri(predaja):
    for datoteka in predaja.datoteke:
        skeniraj(datoteka)          # samo CPU: petlja ne može promijeniti zadatak
        await asyncio.sleep(0)      # dobrovoljna točka prekida, bez I/O
    await pohrana.spremi(predaja)  # čekanje I/O: točka prekida
```

Kod dretvi operacijski sustav prekida izvršavanje između bilo koje dvije strojne instrukcije, a CPython mijenja dretvu između instrukcija bytecodea (zadano svakih 5 ms, vidi `sys.getswitchinterval()`). Zato ni `brojac += 1` nije nedjeljiv: čitanje, zbrajanje i upis zasebni su koraci. U oba slučaja vrijedi isto: **između provjere i upisa može doći na red netko drugi.** Tome se vraćamo kod ponovnih pokušaja.

Ista logika vrijedi na dvije razine (slika 5). Operacijski sustav preemptivno raspoređuje dretve na jezgri. Unutar jedne dretve petlja događaja kooperativno raspoređuje korutine: to je obična petlja u korisničkom prostoru, a ne zaseban proces. Zadatak (`Task`) odgovara dretvi, red spremnih poziva petlje redu spremnih dretvi, a `await` na I/O blokirajućem pozivu: socket se registrira, a korutina se nastavlja kad `epoll` javi da je spreman. Preempcije na razini petlje nema, pa korutina koja ne dođe do `await` zaustavlja sve ostale. Kada operacijski sustav pauzira dretvu s petljom, ne radi ni petlja ni ijedna korutina, pa korutina prekid ne primijeti. Zato je kod između dva `await` nedjeljiv za ostale korutine iste petlje, ali ne i za druge dretve ili procese.



Slika 5: Dva raspoređivača. Gore operacijski sustav preemptivno izmjenjuje dretve T1 i T2; dolje petlja događaja u dretvi T1 mijenja korutine samo na `await`. Dok radi T2, petlja stoji.

Rob Pike razliku izražava ovako: „Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once." (Konkurentnost je bavljenje mnogim stvarima odjednom. Paralelizam je izvršavanje mnogih stvari odjednom.) Vidi *Concurrency is not Parallelism*. Kuhar koji vodi dva jela ilustrira prebacivanje pažnje; dva kuhara mogu raditi istodobno. Analogija se odnosi na aktivnu pažnju, a ne na svaki fizički proces u kuhinji.



Slika 6: Jedan kuhar prebacuje pažnju između dvaju jela; dva kuhara mogu aktivno raditi istodobno.

Preklapanje korisnog posla s udaljenim čekanjem skriva dio čekanja od pozivatelja; ne uklanja mrežnu latenciju i ne jamči veću propusnost.

Zastanite i predvidite: ako posao A čeka pohranu, može li radni proces napredovati s poslom B? Treba li za to dvije procesorske jezgre?

7 Udaljeni zahtjev ima više od povratne vrijednosti

Sučelje poziva funkcije može skrivati udaljenu razmjenu. Ispod njega pozivatelj priprema zahtjev, zahtjev putuje do drugog procesa, taj proces obavlja posao, a odgovor putuje natrag. Za razrađen uvod u tu granicu i pretvorbu između lokalnih vrijednosti i poruka vidi [3, odjeljak 1.3].



Slika 7: Lokalni poziv prenosi upravljanje unutar jednog procesa. Udaljeni zahtjev prelazi mrežnu granicu u oba smjera i unosi neizvjesnost o tome je li posao obavljen kada odgovor ne stigne. I lokalni pozivi mogu zakazati; razlika nije u tome da su nepogrešivi.

Udaljeni poziv ima nekoliko mogućih troškova: pripremu zahtjeva, prijenos, čekanje u redu, obradu i obradu odgovora. Ti troškovi nisu nužno stalni. Usluga koja obično odgovara brzo može zastati, postati preopterećena ili čekati drugu ovisnost.

Razlika se jasno vidi u kodu. Lokalna provjera roka ima dva ishoda, vrijednost ili iznimku, i izvršila se točno jednom:

```
def provjeri_rok(zadatak, sada):
    return sada <= ROKOVI[zadatak]       # KeyError ako zadatak ne postoji
```

Ista provjera kao udaljeni zahtjev mora razlikovati više slučajeva, a jedan od njih nema odgovor:

```
async def provjeri_rok(klijent, zadatak, sada):
    for pokusaj in range(3):
        t0 = time.perf_counter()
        try:
            r = await klijent.get(f"/zadaci/{zadatak}/rok", timeout=2.0)
            r.raise_for_status()
            return sada <= datetime.fromisoformat(r.json()["rok"])
        except httpx.ConnectError:
            pass   # zahtjev nije poslan: smijemo ponoviti
        except httpx.TimeoutException:
            pass   # spor ili srušen? čitanje smijemo ponoviti
        except httpx.HTTPStatusError as e:
            if e.response.status_code < 500:
                raise  # 404: ponavljanje ne pomaže
        except (KeyError, ValueError):
            raise      # drukčiji odgovor: druga verzija usluge
        finally:
            ms = (time.perf_counter() - t0) * 1000
            log.info("rok %s, pokušaj %d: %.0f ms", zadatak, pokusaj + 1, ms)
        await asyncio.sleep(0.2 * 2**pokusaj + random.uniform(0, 0.1))
    raise RuntimeError("usluga zadataka ne odgovara")
```

Uz to treba odlučiti koliko puta ponoviti i koliko čekati između pokušaja: eksponencijalno rastuće čekanje s nasumičnim pomakom (jitter) sprječava da mnogo klijenata istodobno ponovi zahtjev. Zapis trajanja svakog pokušaja (`finally`) pokazuje je li usluga spora ili nedostupna. Ništa od toga lokalni poziv ne treba. Provjera roka samo čita, pa ponovni pokušaj ništa ne mijenja. Kod upisa je drukčije: ako nakon `POST /predaje` istekne čekanje, ne znamo je li predaja spremljena. Tome se vraćamo u odjeljku o tri slijeda iza jednog isteka čekanja.

Lanac poziva: pouzdanost se množi. Predaja rijetko poziva samo jednu uslugu. Provjera roka, podaci o korisniku, pohrana i obavijest već su četiri udaljena poziva, a predaja

uspijeva samo ako uspiju svi. Ako svaki od N uzastopnih koraka uspijeva neovisno s vjerojatnošću p i nema ponovnih pokušaja, cijeli tok uspijeva s vjerojatnošću

$$P_{\text{uspjeh}} = p^N, \qquad P_{\text{barem jedan ispad}} = 1 - p^N.$$

Tablica 1: Vjerojatnost uspjeha toka od N uzastopnih koraka uz neovisne ispade i bez ponovnih pokušaja.

Broj koraka N	$p =$ **99,99 %**	$p =$ **99,9 %**	$p =$ **99 %**	$p =$ **98 %**
1	99,99 %	99,9 %	99 %	98 %
4	99,96 %	99,60 %	96,06 %	92,24 %
10	99,90 %	99,00 %	90,44 %	81,71 %
100	99,005 %	90,48 %	36,60 %	13,26 %

Već uz četiri poziva od 99 % svaka dvadeset i peta predaja negdje zapne. Kroz sto koraka razlika između 99,99 % i 98 % po koraku postaje razlika između 99 % i 13 % za cijeli tok. Obrnuto, cilj za cijeli tok određuje zahtjev za svaki korak: $p_{\min} = P_{\text{cilj}}^{1/N}$. Za 99,9 % kroz 100 koraka svaki korak mora uspjeti u barem 99,999 % slučajeva, odnosno smije pasti otprilike deset puta na milijun poziva. Isto vrijedi za kašnjenje: ako je 1 % odgovora pojedine usluge sporo, a zahtjev čeka odgovore sto usluga, spor je oko 63 % takvih zahtjeva [4]. Pretpostavke su pojednostavljene. Stvarni ispadi često su povezani (zajednička mreža, zajednička baza), a ponovni pokušaj povećava p pojedinog koraka, ali dodaje opterećenje. Tome se vraćamo kod ponovnih pokušaja.

Latencija (latency) opisuje kašnjenje, na primjer vrijeme od slanja zahtjeva do primitka odgovora. **Kapacitet veze** (bandwidth) opisuje koliko podataka veza može prenijeti. **Propusnost** (throughput) je brzina dovršavanja posla, na primjer broj prihvaćenih projekata u sekundi. Veza velikog kapaciteta i dalje može imati veliku latenciju; usluga velike propusnosti i dalje može imati spore pojedinačne zahtjeve. Navedite mjeru i njezine jedinice umjesto da pojmove smatrate zamjenjivima [3, odjeljak 1.2]. Arhiva od 20 MB čini vrijeme prijenosa primjetnim, ali potvrdu mogu odgoditi i provjera, čekanje u redu i pohrana. Veličina datoteke je ilustracija, a ne izmjerena tvrdnja o performansama.



Slika 8: Analogija s dostavom razdvaja vrijeme jedne dostave, kapacitet tereta na ruti i broj dovršenih dostava. To su različite mjere: velik kapacitet tereta sam po sebi ne jamči kratko putovanje ni brzo prihvaćanje na odredištu.

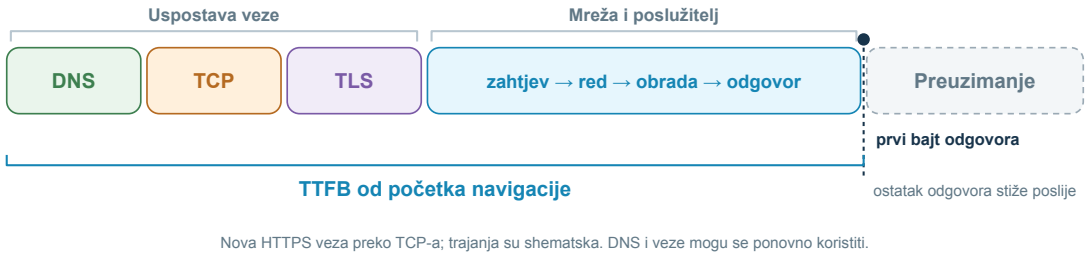
Zamislite da projekt šaljete kao paket. **Latencija** pita koliko traje ova dostava; **kapacitet veze** koliko tereta ruta može prenijeti u sekundi; **propusnost** koliko se dostava stvarno dovrši u sekundi. Količina tereta i broj dovršenih dostava različite su jedinice. U portalu latenciju mjerimo dok rezultat ne stigne do studenta, uključujući odgovor. Propusnost ovdje brojimo kao prihvaćene projekte u sekundi. Nakon brzog prijenosa i dalje se može čekati provjera ili pohrana: dolazak datoteke nije isto što i prihvaćanje projekta.

Pitanje za razmišljanje: kako biste **100 TB podataka između dvaju podatkovnih centara prenijeli što je brže moguće**? Koje informacije trebate prije nego što odaberete način?

Mogući smjer razmišljanja: usporedite slanje podataka mrežom s kopiranjem na diskove i njihovim prijevozom. Završetak definirajte kao trenutak kada su svi podaci kopirani i provjereni na odredištu. Pitajte o udaljenosti, trajnoj brzini mrežnog prijenosa, raspoloživom kapacitetu diskova, brzini kopiranja i vremenu prijevoza. U dekadskim jedinicama 100 TB sadrži 800 bilijuna bitova; uz trajnih 10 Gbit/s sam prijenos traje oko **22,2 sata**, prije dodatnih troškova. Za fizički prijevoz uključite pisanje na diskove, putovanje, čitanje i provjeru. Nijedan način nije uvijek najbrži. Prijevoz diskova može dati veliku efektivnu propusnost za velik skup podataka uz dugu latenciju dostave; bio bi loš način da se dobije brz odgovor na mali zahtjev. Brojevi su ilustracija, a ne izmjerene performanse podatkovnog centra. Natrag na portal: učinkovit prijenos velike količine podataka ne jamči brz odgovor jednom studentu.

7.1 Prisjetimo se Web aplikacija: vrijeme do prvog bajta

Vrijeme do prvog bajta (time to first byte, TTFB) mjeri čekanje do prvog bajta odgovora, a ne do završetka preuzimanja odgovora. Ovdje mjerimo od početka navigacije, uključujući DNS i uspostavu veze kada su potrebni. Mjerenje koje počinje HTTP zahtjevom isključuje taj raniji posao. Uvijek navedite početnu točku. Čekanje uključuje mrežno putovanje, čekanje u redu na poslužitelju i obradu. Vidi web.dev, Time to First Byte.



Slika 9: Pojednostavljena nova HTTPS veza preko TCP-a. TTFB završava prvim bajtom odgovora; slijedi preuzimanje ostatka odgovora. DNS i veze mogu se ponovno iskoristiti. Širine odsječaka su shematske.

Velik kapacitet veze ne uklanja čekanje na posao poslužitelja ni mrežno putovanje. Vrijeme preuzimanja ovisi o veličini odgovora i postignutoj brzini prijenosa. U našem portalu rani bajt odgovora ne dokazuje trajno prihvaćanje: odgovor možda samo potvrđuje posao na čekanju. Skica izostavlja preusmjeravanja, priručnu pohranu, obradu u service workeru i informativne odgovore; HTTP/3 koristi QUIC umjesto prikazane uspostave veze preko TCP-a.

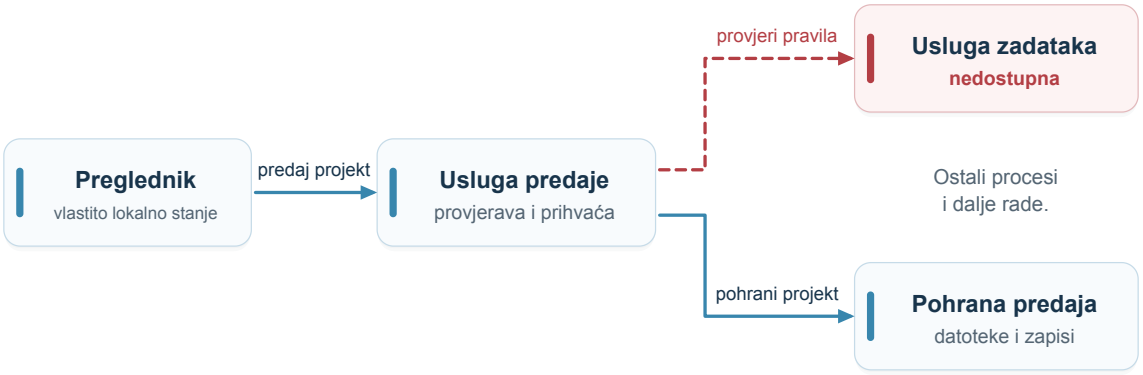
8 Djelomični ispad

Portal se i dalje otvara, ali usluga zadataka je stala. Može li student dohvatiti staru potvrdu? Može li se nova predaja prihvatiti bez provjere roka? Usluga zadataka može prestati odgovarati dok proces predaje, pohrana predaja i preglednik studenta i dalje rade. Taj konkretni kontrast motivira sljedeći pojam.

Definicija 1.4

Djelomični ispad (partial failure) nastaje kada neke komponente prestanu raditi, dok druge nastavljaju.

Komponente ne moraju ispasti zajedno, a proces koji radi i dalje može biti nesposoban dovršiti određenu operaciju [2; 3, odjeljak 2.3].



Slika 10: Usluga zadataka je nedostupna, a ostali procesi i dalje rade. Može li se predaja dovršiti ovisi o informacijama i jamstvima koja operacija traži.

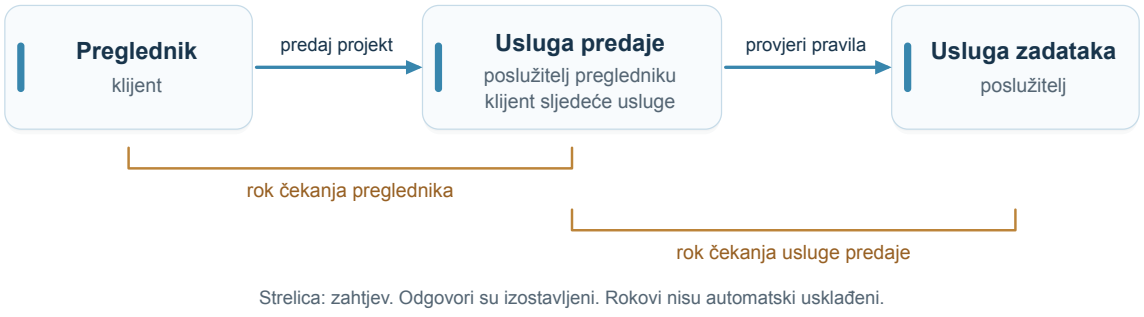
„Poslužitelj radi" i „operacija usluge je dostupna" različite su tvrdnje. Ako svaka predaja mora provjeriti trenutačni rok i pravila datoteka, gubitak usluge zadataka može spriječiti nova prihvaćanja čak i dok prijenosi ili dohvat potvrda i dalje rade. Druga operacija, poput čitanja prethodno spremljene potvrde, možda i dalje radi. O dostupnosti treba govoriti za određenu operaciju i jamstva koja ona traži.

Prekid mreže također može spriječiti komunikaciju dvaju procesa koji rade. Promatrač koji ne dobije odgovor ne može uvijek razlikovati je li proces stao, je li ruta pukla ili odgovor samo kasni. Otkrivanje ispada stoga je povezano s pretpostavkama koje činimo o vremenu.

9 Tri slijeda iza jednog isteka čekanja

9.1 Tko je klijent i zašto istek čekanja?

Klijent je pozivatelj u interakciji; ne mora biti preglednik. Usluga predaje djeluje kao poslužitelj kada prima zahtjev preglednika, a kao klijent kada traži od usluge zadataka da provjeri pravila. Svaki pozivatelj može postavkama aplikacije ili biblioteke odrediti vlastiti rok čekanja, kako ga poziv ne bi ostavio da čeka beskonačno. Ti rokovi nisu automatski usklađeni između slojeva.

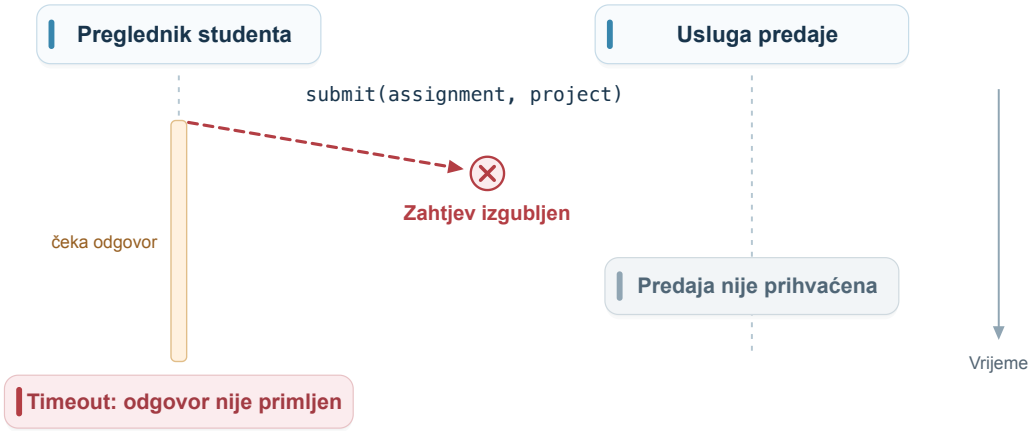


Slika 11: Klijent i poslužitelj su uloge u svakom pozivu. Srednji sloj je poslužitelj pregledniku i klijent usluzi zadataka. Svaki pozivatelj može postaviti vlastiti rok čekanja.

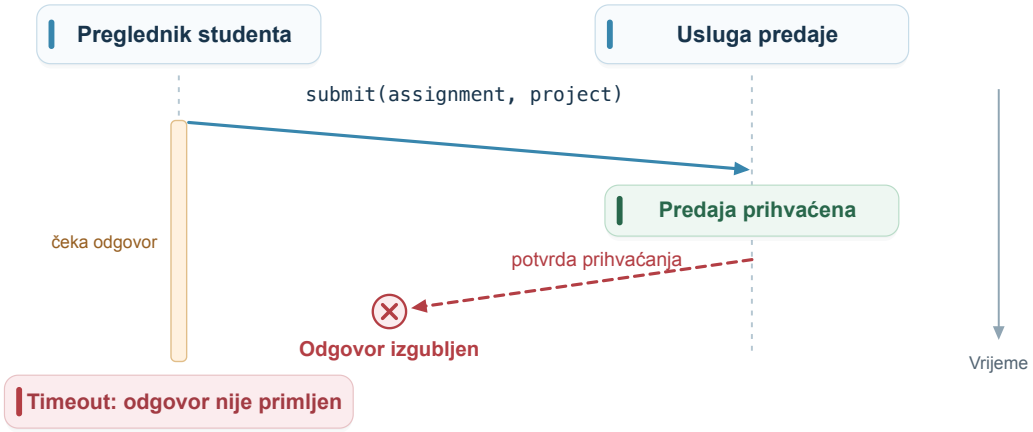
Istek vremena čekanja znači da je pozivatelj odustao od čekanja. Ne uništava zahtjev: on možda još putuje, čeka u redu ili se izvršava. Istek čekanja ni automatski ne otkazuje udaljenu obradu, ne poništava spremljeni učinak i ne utvrđuje ishod. Izričito otkazivanje traži zaseban mehanizam i može se natjecati s dovršetkom. Zahtjev je ovdje operacija na razini aplikacije koja može obuhvatiti mnogo mrežnih paketa. Sljedeći slijedovi kao klijenta koriste preglednik, ali ista se neizvjesnost javlja i između usluga.

9.2 Tri moguća slijeda

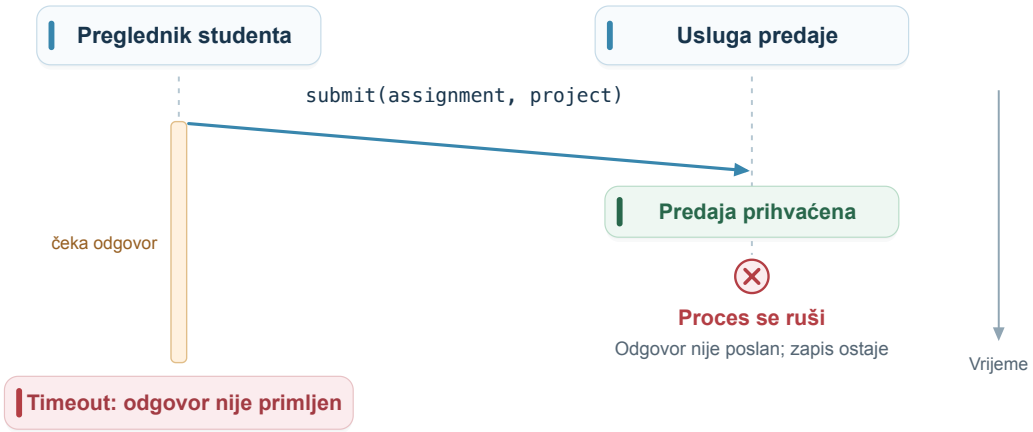
Vratimo se zahtjevu studentice. Uspoređujemo tri slijeda događaja s istim rezultatom vidljivim klijentu. Strelice sažimaju cijelu operaciju predaje, a ne jedan mrežni paket. Slijed A uključuje prijenos koji se nikada ne dovrši; privremeni dijelovi nisu prihvaćene predaje. Slijed B sprema cijelu datoteku i prihvaćeni zapis prije nego što se odgovor izgubi. Slijed C ih trajno sprema prije nego što se proces poslužitelja sruši bez slanja odgovora.



Slika 12: Slijed A: cijeli zahtjev za predaju nikada ne stigne do usluge. Nijedna predaja nije prihvaćena, a klijent na kraju javlja istek čekanja.



Slika 13: Slijed B: usluga prima zahtjev i sprema predaju, ali se odgovor izgubi. Klijent javlja isti istek čekanja kao u slijedu A.



Slika 14: Slijed C: trajno prihvaćanje prethodi rušenju procesa poslužitelja. Odgovor nije poslan i pregledniku istekne čekanje; prihvaćeni zapis ostaje sačuvan.

Slijed C pretpostavlja da ispad procesa ne uništava trajni zapis o predaji. Rušenje prije prihvaćanja moglo bi umjesto toga ostaviti sustav bez prihvaćene predaje. Prikazujemo klijenta koji prije svog roka ne dobije nikakav konačan odgovor; transportni sloj ponekad može ranije prijaviti pogrešku veze.

U sva tri slijeda klijent je poslao zahtjev i nije dobio odgovor prije svog roka. Učinak na poslužitelju je različit. **Istek vremena čekanja je opažanje o čekanju**, a ne konačna tvrdnja da se udaljeni učinak dogodio ili nije dogodio.

To nisu jedini mogući slijedovi. Zahtjev može još putovati, čekati u redu ili biti djelomično obrađen u trenutku isteka čekanja. Primjeri pokazuju zašto samo opažanje ne može razlikovati uspjeh od neuspjeha. Primjenjuju pojam neizvjesnog ishoda iz [3, odjeljak 2.1] na predaju projekta.

9.3 Rasprava: treba li studentica pokušati ponovno?

Ponovni pokušaj (retry) mogao bi pomoći u slijedu A. U slijedovima B i C nezaštićeni ponovni pokušaj mogao bi ponoviti učinak, na primjer stvoriti drugi zapis o predaji ili dvaput obavijestiti nastavnika. Slijed C traži oporavak prije nego što usluga može obraditi ponovni pokušaj. Hoće li doći do dupliciranja ovisi o operaciji i implementaciji; istek čekanja sam po sebi ne znači dupliciranje.

Bolji dizajn pita kako se klijent kroz više pokušaja može pozivati na **istu namjeravanu operaciju**. Na primjer, identifikator zahtjeva može se povezati sa zabilježenim rezultatom. Ponovljeni zahtjev tada se može prepoznati. Ipak, samo generiranje identifikatora nije dovoljno: primatelj ga mora dosljedno koristiti, a rušenja ne smiju ostaviti zabilježen učinak bez informacije potrebne da ga se prepozna.

Semantiku ponovnih pokušaja, idempotentnost i trajno otkrivanje duplikata razvijamo u kasnijim predavanjima. Za sada zadržite pitanje dizajna: **kako aplikacija može razriješiti neizvjestan ishod, a da slučajno ne ponovi učinak?**

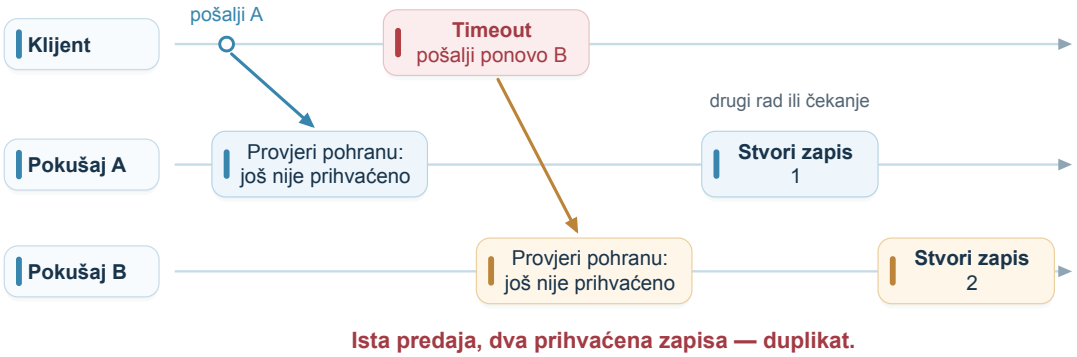
9.4 Konkurentnost i ponovni pokušaji

U raspodijeljenim sustavima nije sve atomarno. U ovom su primjeru provjera pohrane i stvaranje prihvaćenog zapisa zasebni koraci. Između njih usluga može čekati ulaz/izlaz, prepustiti se drugom zahtjevu ili je operacijski sustav može pauzirati. I kratak razmak može omogućiti da drugi pokušaj djeluje na istom ranijem stanju. Atomarnost (atomicity) bi relevantnu operaciju učinila nedjeljivom u odnosu na operacije koje se s njom natječu; mora je osigurati odgovarajući mehanizam, a ne smije se pretpostaviti. Ista utrka „provjeri pa zapiši” može se dogoditi i na jednom računalu.

Razmotrimo još jedan mogući slijed: izvorni zahtjev nije izgubljen, nego je spor ili čeka u redu. Klijent šalje pokušaj A, njegov rok čekanja istekne i šalje ponovni pokušaj B. Poslužitelj sada može raditi na oba pokušaja iste namjeravane predaje. Primamljiva implementacija glasi: „ako prihvaćeni zapis ne postoji, stvori ga”. Što se događa ako oba pokušaja obave provjeru prije nego što ijedan stvori zapis? Sljedeće ispreplitanje otkriva problem.

Istek čekanja ne otkazuje posao koji se već izvršava na poslužitelju. Prisjetite se ranijeg dijagrama konkurentnosti, ali neka sada A i B budu pokušaji iste predaje, a ne radovi različitih studenata. A provjerava postoji li prihvaćena predaja i ne pronalazi je. Prije nego što A zabilježi rezultat, B također provjerava i ne pronalazi je. Ako zatim oba stvore novi zapis o prihvaćenoj predaji, nastavnik vidi dva zapisa za istu namjeravanu predaju. Ponovni pokušaj može se preklopiti s izvornim pokušajem čak i nakon što je pregledniku isteklo čekanje.

Za ovaj slijed nije potrebno istodobno izvršavanje. Ispreplitanje na jednom radnom procesu može otkriti isti logički problem kada operacija obuhvaća više koraka. Zato razmišljamo o dopuštenim ispreplitanjima i zajedničkom stanju, umjesto da pretpostavimo kako brz procesor ili jedna instanca usluge uklanjaju problem.



Slika 15: Klijentu istekne čekanje i šalje ponovni pokušaj dok pokušaj A traje. Oba pokušaja provjeravaju spremljene predaje prije nego što ijedan zabilježi prihvaćanje, a zatim stvaraju dva prihvaćena zapisa za jednu namjeravanu predaju.

10 Invarijante i napredak

Nastavnik nakon ponovnog pokušaja vidi dva prihvaćena zapisa. Koje je pravilo to trebalo spriječiti? Treba nam uvjet koji vrijedi u svakom dopuštenom ispreplitanju, uključujući dva pokušaja koji se preklapaju.

Definicija 1.5

Invarijanta (invariant) je pravilo koje mora vrijediti u svakom dopuštenom stanju sustava.

Za primjer projekta korisna je invarijanta: „ponovni pokušaji iste namjeravane predaje stvaraju najviše jedan prihvaćeni zapis". To pravilo biramo za ovaj primjer. Namjerno prerađen projekt može biti nova verzija prema izričitom pravilu o ponovnoj predaji; to je nešto drugo od ponavljanja iste operacije nakon isteka čekanja.

Pretpostavimo da dupliciranje riješimo odbijanjem svake predaje. Sada izbjegavamo duplikate, ali studenti nikada ne mogu dovršiti svoj posao. Koristan portal treba i zaštitu od loših ishoda i obećanje napretka pod prikladnim uvjetima.

Definicija 1.6

Svojstvo sigurnosti (safety) zahtijeva da se određeni loš događaj ili stanje nikada ne dogodi.

Na primjer, usluga nikada ne smije prihvatiti dva zapisa za istu namjeravanu predaju. Održavanje invarijante o jednom zapisu podupire ovo svojstvo sigurnosti. Svojstvo sigurnosti (safety) ovdje znači ispravnost, a ne zaštitu od napadača (security).

Definicija 1.7

Svojstvo napredovanja (liveness) zahtijeva da se određeni napredak konačno ostvari pod navedenim pretpostavkama.

Na primjer, valjana predaja na kraju biva prihvaćena, pod uvjetom da se potrebne usluge oporave i ostanu dostupne, da komunikacija na kraju uspije, da se posao na čekanju sačuva ili ponovi i da se posao u redu na kraju obradi. Valjanost slijedi primjenjiva pravila zadatka, uključujući pravilo o roku. To je jamstvo napretka, a ne obećanje određenog vremena odgovora. Uz svaku tvrdnju o ispravnosti uvijek treba navesti točno svojstvo i pretpostavke.

Usluga može izbjeći duplikate tako da odbije svaku predaju, ali to nije korisno potpuno rješenje. Može brzo odgovarati tako da za svaki zahtjev stvori zapis, ali tada ponovni pokušaji mogu stvoriti duplikate. Moramo navesti i što ne smije poći po zlu i uvjete pod kojima se koristan posao mora dovršiti.



Slika 16: Dva različita neuspjeha: dvostruko prihvaćanje i izostanak konačnog prihvaćanja iako pretpostavke napretka vrijede.

Prije odabira protokola zapišite invarijantu, dopuštene ispade i dokaze dostupne svakom sudioniku. Te tri stavke čine raspravu o arhitekturi mnogo preciznijom od popisa tehnologija.

10.1 Sigurnost i napredovanje u različitim sustavima

Sustav	Primjer tvrdnje o sigurnosti	Primjer tvrdnje o napredovanju
Plaćanje	Ponovni pokušaji nikada ne terete kupca dvaput za isto namjeravano plaćanje.	Valjano odobreno plaćanje na kraju se dovršava.
Rezervacija sjedala	Dva kupca nikada nemaju potvrđenu rezervaciju istog sjedala za isti događaj.	Svaki zahtjev na kraju dobiva potvrdu ili odbijenicu.
Red poslova	Posao se nikada ne označava dovršenim prije nego što uspije.	Svaki valjani posao na čekanju na kraju se obradi.

Sustav	Primjer tvrdnje o sigurnosti	Primjer tvrdnje o napredovanju
Raspodijeljeno zaključavanje (distributed lock)	Najviše jedan proces istodobno izvršava zaštićenu kritičnu sekciju.	Svaki ispravan proces koji nastavlja tražiti zaključavanje na kraju ga dobiva.

To su tvrdnje koje bismo mogli zahtijevati, a ne svojstva koja sustav automatski daje samim svojim imenom. Napredak traži odgovarajući oporavak usluga, uspješnu komunikaciju, sačuvan ili ponovljen posao i pravedno raspoređivanje. Poslovi moraju završiti ili doći do definiranog ishoda neuspjeha. Tvrdnja o zaključavanju je odsutnost izgladnjivanja (starvation freedom) i dodatno traži pravedno dodjeljivanje, konačne kritične sekcije i siguran oporavak nakon ispada vlasnika zaključavanja. Odbijenica rezervacije zadovoljava navedenu tvrdnju o odgovoru; ne obećava svima sjedalo. Sigurnost reda ovdje ne tvrdi izvršavanje točno jednom (exactly-once). Kao i prije, konačni napredak ne daje brojčanu granicu vremena odgovora.

11 Navedite model sustava

Možemo li obećati da će svaka valjana predaja na kraju biti prihvaćena ako struja ostane isključena, disk ispadne bez preostale obnovljive kopije ili prerezani kabel ostavi uslugu trajno odspojenom? Ti primjeri dovode u pitanje napredovanje: zaustavljen sustav može izbjeći dvostruko prihvaćanje, ali ne može napredovati. Privremeni ispadi mogu se podnijeti kada se napajanje i povezanost oporave, a posao preživi ili se ponovi. Ispad diska ne mora izgubiti posao ako replika, sigurnosna kopija ili kopija na klijentu omogućuju oporavak; primjer pretpostavlja da takav oporavak ni ponovna predaja više nisu mogući. Naše tvrdnje ovise o pretpostavkama o napajanju, pohrani, komunikaciji i oporavku. Sljedeće pitanje glasi: **uz koje ispade i kašnjenja sustav može pružiti svojstva koja tvrdi?**

Model sustava (system model) izričito navodi pretpostavke o čvorovima koji sudjeluju, komunikaciji i vremenu. To je apstrakcija za razmišljanje o ponašanju, a ne crtež svake pojedinosti implementacije [3, odjeljak 2.3].

Na primjer, portal možemo modelirati tako da dopušta zakašnjele ili izgubljene poruke, uslugu predaje koja se može srušiti i ponovno pokrenuti te nikakvo zajamčeno najdulje vrijeme odgovora. Algoritam iznosi **tvrdnje o ispravnosti**, a svaka tvrdnja mora vrijediti u svakom izvođenju dopuštenom navedenim pretpostavkama modela. Primjer tvrdnje: **ponovni pokušaji nikada ne stvaraju dva prihvaćena zapisa iste predaje, čak ni nakon ponovnog pokretanja usluge**. To svojstvo treba opravdati algoritmom i njegovim pretpostavkama; sam model ga ne daje.

Tvrdnja o napretku, poput „valjana predaja na kraju biva prihvaćena”, traži dodatne pretpostavke: usluge se oporavljaju i ostaju dostupne, komunikacija na kraju uspijeva, posao na čekanju preživljava ili se ponavlja, a obrada se na kraju izvršava. Kada bismo dopustili da se poruke gube zauvijek, takvo jamstvo napretka ne bi bilo moguće.

11.1 Pretpostavke o mreži

Infrastrukturne pretpostavke razrađujemo kroz ponašanje mreže, ponašanje čvorova i vrijeme. Počnimo prerezanim kabelom: oporavlja li se komunikacija ili ostaje prekinuta? Ponovni pokušaj koristan je samo ako neki kasniji pokušaj može stići do primatelja. To je mrežni dio modela; ispadi čvorova i vremenske granice slijede u nastavku.

Što ako se svaki ponovni pokušaj izgubi? Ponavljanje izgubljenog zahtjeva korisno je samo ako neki pokušaj na kraju može proći. Taj uvjet moramo navesti, a ne zaključiti ga iz samog čina ponavljanja.

Pretpostavka	**Značenje za primjer**
Poruke se mogu izgubiti, udvostručiti ili stići drugim redoslijedom	Obraditi izostale odgovore, ponovljene zahtjeve i različit redoslijed dolaska.
Uporni ponovni pokušaji između ispravnih krajnjih točaka na kraju prolaze	Konačna dostava postaje moguća, ali iz toga ne slijedi rok dostave.
Mrežna particija trajno blokira komunikaciju	Posao koji traži tu komunikaciju ne može se zajamčeno dovršiti.

Veza s **poštenim gubitkom** (fair-loss link) dopušta gubitak, ali jamči konačnu dostavu uz uporno ponovno slanje između ispravnih krajnjih točaka. **Pouzdana** veza (reliable link) jamči konačnu dostavu bez udvostručavanja i izmišljanja poruka; ne mora čuvati redoslijed poruka. **Proizvoljna** veza (arbitrary link) dopušta i napadača koji mijenja ili izmišlja poruke. Ti modeli veza opisuju dopušteno ponašanje, a ne granicu kašnjenja [3, odjeljak 2.3]. Njihove implementacije u ovom predavanju ne razvijamo.

Mrežna particija (network partition) sprječava komunikaciju između nekih sudionika iako oni i dalje rade. Može biti privremena ili trajna. Ponovni pokušaji aplikacije i udvostručavanje u mreži dva su različita razloga zbog kojih primatelj može dobiti ponovljene zahtjeve. Oba su važna kada štitimo pravilo o jednom zapisu.

U našem primjeru zahtjevi i odgovori mogu kasniti ili se izgubiti. Primjer ne sadrži aktivnog napadača. To su izbori modela, a ne univerzalne činjenice o svakom razmještaju. Ako kasnije pretpostavimo da ponovno slanje na kraju uspijeva, moramo to reći; trajna particija proturječila bi toj pretpostavci.

Jamstvo dostave i vremensko jamstvo različite su stvari. „Na kraju dostavljeno” ne znači „dostavljeno unutar dvije sekunde”. Iz tvrdnje koja obećava samo konačan napredak ne smijemo zaključiti rok.

11.2 Pretpostavke o čvorovima

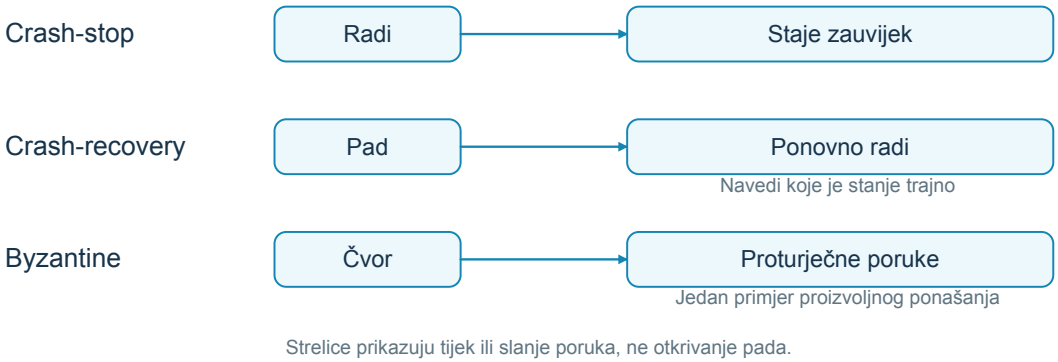
Usluga stane nakon što prihvati projekt. Što se događa kada se vrati? Najprije pitajte koje stanje preživljava. Prihvaćeni zapis u trajnoj pohrani i posao na čekanju koji postoji samo u memoriji različito se ponašaju pri oporavku. Zatim razlikujte ponovno pokretanje, trajni prestanak rada i potpuno odstupanje od algoritma.

Model	Što se smije dogoditi	Posljedica za razmišljanje
Ispad bez oporavka (crash-stop)	Sudionik unutar modela trajno staje; oporavak se ne pretpostavlja.	Algoritam se mora snaći, a da se ne oslanja na njegov povratak.
Ispad s oporavkom (crash-recovery)	Čvor se nakon rušenja može ponovno pokrenuti.	Navedite koje je stanje trajno, a koje se stanje u memoriji gubi.
Bizantski model (Byzantine)	Neispravan čvor može proizvoljno odstupiti od protokola.	Samo preživljavanje rušenja nije dovoljna zaštita.

Na primjer, radni proces umre usred izračuna, a preostali radni procesi moraju dovršiti posao bez njega. Ispad bez oporavka isključuje daljnje korake tog sudionika u izvođenju koje analiziramo; ne tvrdi da se njegovo fizičko računalo nikada ne može ponovno pokrenuti. Ako se isti sudionik ponovno pokrene i vrati, koristite model ispada s oporavkom i navedite koje stanje preživljava. Dovršavanje posla unatoč ispalom radnom procesu zahtjev je za koji treba dizajnirati, a ne automatsko jamstvo modela.

Te kategorije opisuju dopušteno ponašanje, a ne koliko se brzo ispad otkriva. Posebno, pretpostavka ispada bez oporavka ne daje drugim čvorovima automatski trenutačno saznanje o rušenju. Bizantski dogovor u ovom uvodnom predavanju ne analiziramo.

U portalu **ispad bez oporavka** može značiti da proces predaje staje i više se ne pokreće tijekom izvođenja koje analiziramo. Kod **ispada s oporavkom** on se ponovno pokreće: pretpostavimo da prihvaćeni zapisi preživljavaju u trajnoj pohrani, ali red čekanja u memoriji nestaje. **Bizantska** usluga mogla bi studentici javiti „prihvaćeno”, a nastavniku „nije prihvaćeno”. Proturječni odgovori jedan su primjer proizvoljnog odstupanja, koje ne mora biti zlonamjerno.



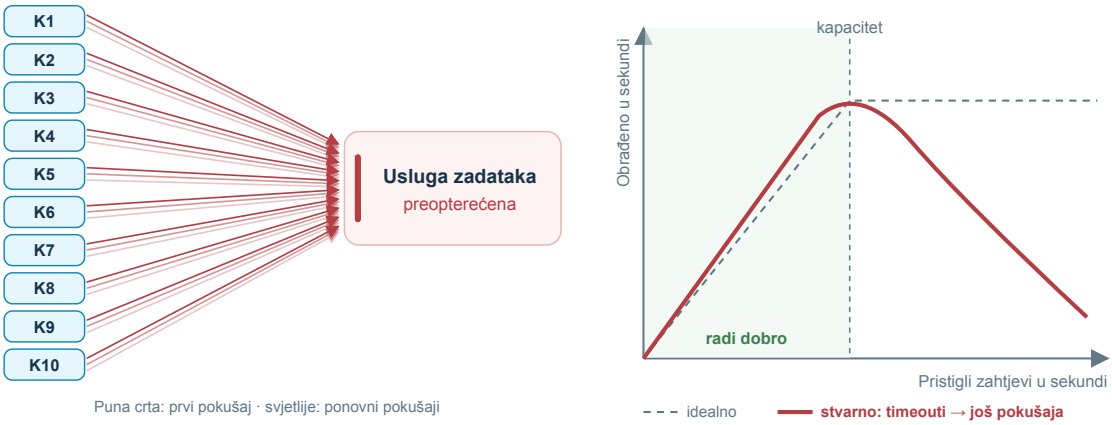
Slika 17: Primjeri dopuštenog ponašanja u trima modelima ispada čvorova. Strelice ne znače da drugi čvor otkriva rušenje.

11.3 Zašto su vremenske pretpostavke važne

Usluga zadataka obično odgovara brzo. Danas šuti. Koliko dugo portal treba čekati? Treba li pokušati ponovno, nastaviti čekati ili studentici reći da je ishod nepoznat? „Mi" ovdje znači arhitekti i razvojni inženjeri portala: ljudi koji odabiru i implementiraju pravila čekanja i ponovnih pokušaja. Pokrenuta usluga slijedi ta pravila. Odabir roka čekanja ne stvara jamstvo vremena odgovora. Vremenske pretpostavke određuju što izostanak odgovora dokazuje [3, odjeljak 2.3].

Bez zajamčene granice vremena odgovora šutnja ne može razlikovati sporu uslugu od srušene. Istek čekanja i dalje može pokrenuti radnju oporavka, ali izvorni zahtjev može ostati aktivan. Ponovni pokušaj stoga mora čuvati invarijantu bez duplikata čak i kada se oba pokušaja preklapaju. To model izravno povezuje sa **sigurnošću**.

Uvid: pravila čekanja i ponovnih pokušaja mogu preopteretiti uslugu kojoj trebaju pomoći. Ako mnogo klijenata koristi prekratke rokove čekanja i agresivno ponavlja zahtjeve, njihovi izvorni zahtjevi možda se još izvršavaju kada stignu novi pokušaji. Veće opterećenje usporava odgovore, a to pokreće još ponovnih pokušaja: nastaje **oluja ponavljanja** (retry storm). Legitimni raspodijeljeni klijenti tako mogu izazvati samouzrokovanu uskratu usluge s učincima poput DDoS-a, bez zlonamjernog napadača. Ograničite broj ili budžet ponovnih pokušaja i koristite odgodu s rastućim razmakom (backoff) uz nasumični pomak (jitter) kako biste smanjili pojačavanje i usklađene navale. Te mjere kontroliraju opterećenje; ne zamjenjuju sigurnu obradu ponovljenih zahtjeva. Vidi AWS Builders' Library, *Timeouts, retries, and backoff with jitter*.



Slika 18: Oluja ponavljanja. Lijevo deset klijenata s prvim i dva ponovna pokušaja šalje zahtjeve istoj preopterećenoj usluzi. Desno shematska krivulja: do kapaciteta broj obrađenih zahtjeva raste, a iznad kapaciteta pada jer timeouti izazivaju još pokušaja.

Postoji i posljedica za **napredovanje**. Beskonačno čekanje izbjegava preklapanje s ponovnim pokušajem, ali može ostaviti valjan posao nedovršenim. Konačno prihvaćanje traži izričite uvjete: potrebne se usluge oporavljaju i ostaju dostupne, komunikacija na kraju uspijeva, posao na čekanju se čuva ili ponavlja, a obrada se na kraju izvršava. Ti uvjeti podupiru argument o napretku; ne daju fiksni rok odgovora.

Ako model jamči da svaka usluga koja radi odgovara unutar poznate granice s kraja na kraj, uz pouzdanu komunikaciju i ostale pretpostavke, prekoračenje te granice isključuje ispravno izvođenje. Sama granica mrežnog kašnjenja nije dovoljna: i obrada i povratni put traju. Kada se poruke mogu izgubiti ili mreža može ispasti neovisno, šutnja ne može jednoznačno otkriti rušenje usluge.

U modelu konačne sinkronosti (eventual synchrony) sumnje na temelju isteka čekanja mogu biti pogrešne prije stabilizacije. Kada granice počnu vrijediti, rok čekanja dulji od relevantne granice odgovora može izbjeći lažne sumnje uzrokovane kašnjenjem, uz preostale pretpostavke. Sudionici ne znaju kada nastupa stabilizacija, a vremenska pretpostavka sama po sebi ne jamči da proizvoljan algoritam napreduje. I dalje moramo objasniti kako je algoritam koristi.

Tvrdnja poput „sustav će na kraju prihvatiti predaju” mora, dakle, navesti pretpostavke pod kojima vrijedi. Sljedeći modeli izričito navode vremenski dio tih pretpostavki.

11.4 Vremenske pretpostavke

U **sinkronom modelu** prikladne poznate granice ograničavaju kašnjenja komunikacije i obrade. U **asinkronom modelu** algoritam se ne smije oslanjati na takve vremenske granice. Čest **djelomično sinkroni** model (partially synchronous) dopušta početno razdoblje bez

upotrebljivih vremenskih granica, nakon kojega slijedi razdoblje u kojemu granice vrijede; algoritam možda ne zna kada to stabilno razdoblje počinje.

Zašto djelomična sinkronost? Zagušenja Interneta, promjene usmjeravanja i pauze u izvršavanju procesa motiviraju model koji dopušta nepredvidiva kašnjenja [3, odjeljak 2.3]. Slika koristi formulaciju **konačne sinkronosti**: nakon nepoznatog konačnog trenutka stabilizacije granice vrijede do kraja izvođenja. To je pretpostavka za razmišljanje, a ne jamstvo Interneta ni tvrdnja da je 99 % zahtjeva brzo. Stvarni sustavi mogu više puta postati nestabilni; dovoljno dug stabilan interval može omogućiti dovršetak operacije, ovisno o algoritmu i njegovim ostalim pretpostavkama. Koristan cilj dizajna je **sigurnost unatoč nepredvidivom vremenu i napredak kada se uvjeti stabiliziraju**. Algoritam mora utvrditi obje tvrdnje; odabir modela ih ne dokazuje.

Ovaj rječnik opisuje jamstva. Ne znači isto što i blokirajući ili neblokirajući API. Program koji koristi `async` i `await` time ne dokazuje da njegov raspodijeljeni algoritam koristi asinkroni model sustava.

11.5 Primjeri s ilustrativnim brojevima

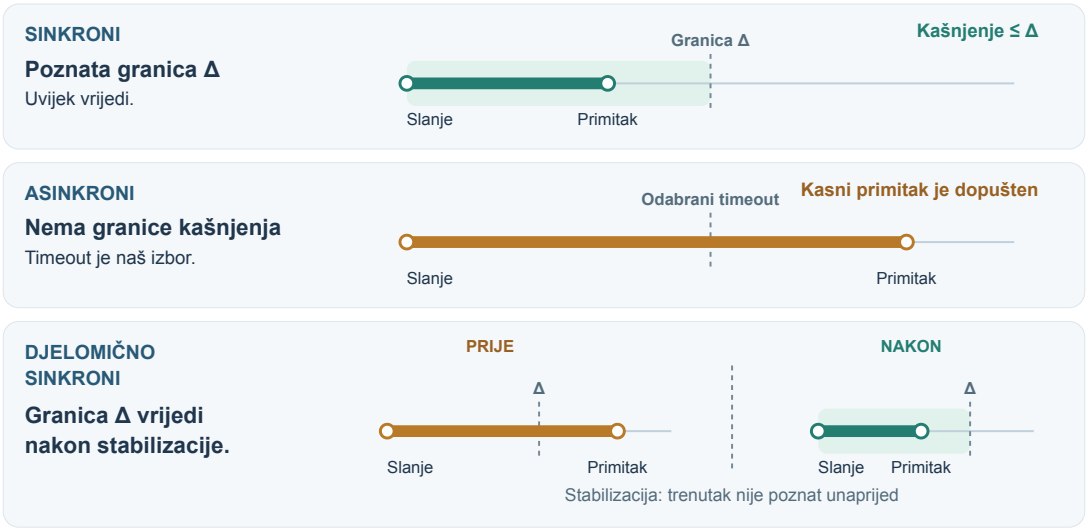
Sljedeći brojčani primjeri ilustriraju vremenske pretpostavke neovisno o razmještaju:

- **Sinkroni:** pretpostavimo da svaka poruka između ispravnih čvorova stiže unutar 20 ms. Dolazak nakon 35 ms krši tu pretpostavku. Sama tipična brzina ne opravdava takvu granicu.
- **Asinkroni:** za prijenos preko Interneta ne pretpostavljamo granicu kašnjenja. Odgovor koji stigne nakon 2 s dopušten je čak i ako je klijent odabrao rok čekanja od 1 s.
- **Djelomično sinkroni:** zagušenje isprva uzrokuje kašnjenja od 2 s. Nakon nepoznatog trenutka stabilizacije pretpostavljamo da sve nove poruke između ispravnih čvorova stižu unutar 20 ms i da ta granica nastavlja vrijediti. Kratko razdoblje brzih odgovora nije dovoljno da bi se ta pretpostavka utvrdila.

To su zamišljeni izbori modela, a ne mjerenja ni tvrdnje o produkcijskom portalu. Granica kašnjenja poruke samo je dio granice vremena odgovora: važni su i obrada i povratni put.

Rok čekanja pravilo je koje odabire aplikacija. Može ograničiti koliko korisnik čeka čak i kada ne može dokazati da se udaljeni čvor srušio. Ako model daje pouzdane granice, mogući su jači zaključci. Bez tih granica „obično brzo” nije isto što i „zajamčeno odgovara do tog trenutka”.

Zašto „obično” ne utvrđuje jamstvo? Dopušta iznimke, a jamstvo modela mora vrijediti u svakom izvođenju koje taj model dopušta. Pretpostavimo da odgovori obično stižu unutar 100 ms, ali jedan traje 500 ms. Proglasiti poslužitelj srušenim nakon 100 ms moglo bi spori poslužitelj zamijeniti ispalim. Vrijednosti su ilustrativne. Argument o ispravnosti koji ovisi o gornjoj granici od 100 ms ne pokriva ovo sporije izvođenje; opažanja tipične brzine ne utvrđuju tu granicu. Pretpostavka modela temelj je za razmišljanje, a ne dokaz da je stvarni sustav uvijek zadovoljava.



Slika 19: Vremena slanja i primanja uspoređena s granicom kašnjenja Δ. Odabrani rok čekanja nije jamstvo. Kod konačne sinkronosti poruke poslane nakon nepoznatog trenutka stabilizacije poštuju granicu.

11.6 Primjena pojmova na stvarne sustave

Ovi primjeri uspoređuju određene operacije i konfiguracije, umjesto da cijelom proizvodu pripišu jedno univerzalno jamstvo. Replika je drugi proces koji održava kopiju podataka. Kvorum (quorum) ovdje znači većinu relevantnih članova ili replika.

11.6.1 etcd: atomarno stvaranje ključa

etcd je replicirana pohrana parova (ključ: vrijednost) u kojoj raspodijeljeni sustavi (npr. Kubernetes) drže konfiguraciju i podatke za koordinaciju. Razmotrimo tri člana koji slijede protokol (ne lažu), a mogu se srušiti i oporaviti uz netaknuto trajno stanje protokola. Komunikacija može kasniti ili biti podijeljena particijom. Raft čuva dogovor unatoč tim vremenskim problemima; napredak traži dostupnu većinu te dovoljno stabilnu komunikaciju i obradu. U particiji $2 + 1$ strana s dva člana može izabrati vođu i potvrđivati operacije, a izolirani član ne može potvrditi suprotnu povijest. Gubitak većine zaustavlja pisanja umjesto da dopusti proturječne potvrde. Ti uvjeti slijede model Rafta i etcd dokumentaciju o ispadima.

Invarijanta aplikacije: najviše jedan klijent uspješno stvara ključ prije nego što se taj ključ izbriše. Stvaranje ključa implementirajte kao jednu atomarnu transakciju: usporedi da ključ ne postoji, zatim ga stvori. Pretpostavite da nikakvo brisanje, istek ni drugi kod ne zaobilaze to pravilo. Invarijanta slijedi iz atomarnog redoslijeda transakcije; zasebni pozivi čitanja i pisanja to ne osiguravaju. Ovo primjenjuje jamstva etcd KV API-ja. To ne daje isključivu kontrolu nad vanjskim računalom ili resursom. Kao i prije, klijent kojemu je isteklo čekanje možda ne zna je li njegov upis potvrđen.

11.6.2 Cassandra: konfiguracija mijenja obećanje

Apache Cassandra je raspodijeljena NoSQL baza koja iste podatke čuva na više poslužitelja. Uzmimo tri replike jedne ćelije i obična čitanja i pisanja na **razini konzistentnosti** (consistency level) **ONE**. Koordinatoru treba da jedna relevantna replika potvrdi operaciju. Uz dovoljno resursa klijent stoga može dovršiti operaciju bez kontaktiranja većine, ali kasnije čitanje s druge replike može vratiti stare podatke. Replike slijede protokol (ne lažu), a mogu se srušiti i oporaviti; zakašnjele poruke i particije su očekivane. Usklađivanje na temelju vremenskih oznaka bira između proturječnih vrijednosti, pa odabrana vrijednost nije nužno posljednje pisanje u stvarnom vremenu. Vidi Cassandrinu Dynamo arhitekturu. Razlika od etcd-a je u trenutku potvrde: etcd potvrđuje upis tek kad ga prihvati većina i odredi mu mjesto u jednom redoslijedu, pa u particiji manjina ne može pisati. Cassandra na razini ONE potvrđuje nakon jedne replike i usklađuje replike kasnije, pa mogu pisati obje strane particije, a sukobe rješava vremenska oznaka.

Invarijanta aplikacije koja nije osigurana: „mjesto se prodaje najviše jednom" ne slijedi iz običnih operacija čitaj-pa-piši. Dva klijenta mogu pročitati „slobodno" i oba javiti uspješnu prodaju. Usklađivanje na jednu konačnu spremljenu vrijednost ne poništava ta dva obećanja. Cassandra nudi lake transakcije (lightweight transactions) za uvjetne operacije s jačom konzistentnošću. Sam odabir razine QUORUM ne čini zasebna čitanja i pisanja atomarnima.

Napredak: operacija na razini ONE treba jednu relevantnu repliku koja odgovara; na razini QUORUM treba dvije od tri. Nijedna ne radi kada se ne može doći ni do jedne potrebne replike. Konačna konvergencija traži oporavljenu komunikaciju, sačuvana ažuriranja i učinkovit popravak; kada pisanja prestanu, replike mogu konvergirati nakon što saznaju ista ažuriranja. Hints i repair objašnjavaju zašto je oporavak posao koji sustav i operateri moraju obaviti, a ne nešto što podrazumijeva riječ „konačno" (eventual). Trajni gubitak podataka ili trajna particija poništavaju ovaj argument o konvergenciji.

11.6.3 Bitcoin: valjanost i vjerojatnosna konačnost

Bitcoin koordinira povijest plaćanja među sudionicima koji mogu biti zlonamjerni. Za razliku od gornjeg modela rušenja pouzdanih članova, njegovo sigurnosno razmišljanje uključuje rudarenje koje provodi napadač. Pretpostavlja sigurnu kriptografiju, većinski pošten hash-rate (većinu računalne snage rudarenja drže pošteni rudari) i dovoljno dobru propagaciju blokova u odnosu na rudarenje. Nije dovoljno brojati poštena računala. Čvorovi provjeravaju transakcije i biraju valjani lanac s najviše akumuliranog **dokaza rada** (proof of work). Vidi Bitcoin whitepaper, osobito odjeljke 5 i 11, te vodič za razvojne inženjere.

Invarijanta: izlaz transakcije ne može se potrošiti dvaput unutar jednog valjanog lanca. To ne znači da se svi čvorovi odmah slažu oko jedne povijesti: čvorovi privremeno mogu pratiti suprotstavljene grane koje sadrže proturječne potrošnje. **Sigurnost namire je vjerojatnosna:** dodatne potvrde smanjuju rizik poništenja uz sigurnosne pretpostavke, ali ne utvrđuju apsolutnu konačnost.

Napredak: rudarenje i komunikacija omogućuju rast lanca. Uključivanje određene transakcije dodatno traži da ona ostane valjana, dosegne rudare i bude odabrana; važni su kapacitet, naknade i pravila rudara. Nema fiksnog roka uključivanja. Tijekom particije zasebne grane mogu rasti; nakon ponovnog povezivanja prelazak na odabrani lanac može poništiti prethodno opaženu potvrdu. Napadač s većinom može ugroziti namiru ili cenzurirati transakcije, ali ne može natjerati pošteni čvor koji provjerava transakcije da prihvati nevaljani potpis.

Ovi primjeri pokazuju tri različita obećanja: atomarno stvaranje ključa koje staje bez većine; podesivu potvrdu replika i konačno usklađivanje; te dogovor uz napadače s vjerojatnosnom namirom. U svakom od njih aplikacija i dalje mora navesti vlastitu invarijantu i uvjete napretka.

11.7 Pretpostavke našeg portala zajedno

Model čine razumne pretpostavke koje mi kao arhitekti i razvojni inženjeri portala možemo predvidjeti: one opisuju kvarove na koje dizajn mora računati, a ne činjenice o sustavu, i dizajn vrijedi samo dok one vrijede. Dimenzije zajedno čine jedan model. Nisu uzastopne razine kroz koje sustav mora proći.

- **Mreža:** zahtjevi i odgovori mogu kasniti, izgubiti se, ponoviti ili stići drugim redoslijedom; mijenjanje poruka nije uključeno.
- **Čvorovi:** usluge se mogu srušiti i ponovno pokrenuti. Prihvaćeni zapisi u trajnoj pohrani (diskovi u RAID-u) preživljavaju modelirana rušenja procesa; stanje u radnoj memoriji (RAM) gubi se. To je pretpostavka o pohrani, a ne obećanje da diskovi u stvarnosti ne mogu ispasti.
- **Vrijeme:** ne postoji zajamčeno najdulje vrijeme odgovora.

Željena tvrdnja o sigurnosti glasi: ponovni pokušaji nikada ne stvaraju dva prihvaćena zapisa za istu namjeravanu predaju, uključujući ponovna pokretanja. Implementacija mora opravdati tu tvrdnju: sami trajni zapisi ne usklađuju otkrivanje duplikata s primjenom učinka.

Za konačno prihvaćanje dodatno pretpostavite da potrebne usluge na kraju ostanu dostupne, da komunikacija na kraju uspije, da se posao na čekanju čuva ili ponavlja i da se obrada na kraju izvrši. Trajna particija krši te uvjete napretka, ali ne smije dopustiti dvostruko prihvaćanje. Konačna sinkronost može biti potrebna argumentu o napretku određenog algoritma; nije potrebna za svaki mogući oblik konačnog napretka.

11.8 Dodatno čitanje: modeli sustava u praksi

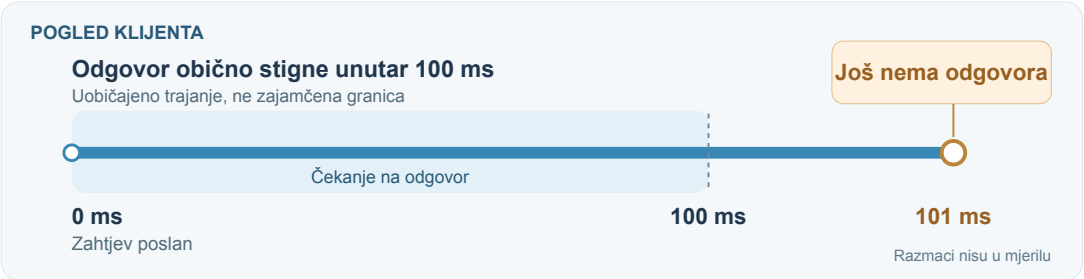
Mjesto razmještaja ne određuje jednoznačno model sustava. Za **WWW aplikaciju** koristan je početak asinkrono kašnjenje, prekidi veze i ispad s oporavkom za pouzdane poslužitelje; klijenti mogu biti nepouzdani i ponašati se proizvoljno. Tvrdnja o ispravnosti pozadinskog sustava mora navesti koje sudionike obuhvaća. Sami nepouzdani klijenti ne znače da pozadinskom sustavu treba bizantski konsenzus.

Za API i radni proces razmještene na **jednom računalu** modelirajte zasebne procese koji koriste IPC ili lokalnu petlju (loopback), moguća ponovna pokretanja procesa i povezani ispad kada ispadne računalo. Običan operacijski sustav ne daje strogu granicu vremena odgovora samo zato što je komunikacija lokalna. Takav razmještaj ne podnosi ispad fizičkog računala.

Za usluge u **lokalnoj mreži (LAN)** malo tipično kašnjenje ne uklanja pauze raspoređivanja, zagušenja ni mrežne particije. Asinkrono vrijeme i ispad s oporavkom za pouzdane poslužitelje razuman su početak. Kada algoritmu za napredak treba stabilno vrijeme, navedite konačnu sinkronost i oporavak kao dodatne pretpostavke. To vrijedi i za web-usluge. Zajamčena sinkronost traži opravdane granice mreže i obrade, a ne samo blizak poslužitelj. To su primjeri izbora modela prema razlikama iz [3, odjeljci 1.1 i 2.3].

12 Zadaci

1. **Prepoznajte granicu.** Na dijagramu predaje navedite svakog sudionika i jedan dio stanja koji posjeduje. Koji bi ispadi računala mogli utjecati na više od jednog sudionika ako dvije usluge dijele isto računalo?
2. **Klasificirajte izvođenje.** Radni proces W izvršava A1, B1, A2, B2. Jesu li A i B konkurentni? Pokazuje li ovaj trag paralelizam? Objasnite razliku.
3. **Protumačite dokaz.** Zahtjevu za predaju istekne čekanje nakon dvije sekunde. Navedite dva različita slijeda na poslužitelju koja odgovaraju tom opažanju. Uklanja li dulji rok čekanja dvosmislenost u modelu bez poznate gornje granice kašnjenja?
4. **Provjerite pretpostavku.** U dizajnu piše: „Usluga zadataka obično odgovara unutar 100 ms, pa se nakon 101 ms sigurno srušila.” Prepoznajte jamstvo koje nedostaje i predložite točniju tvrdnju.



Slika 20: Opažanje klijenta nakon 101 ms. Osjenčano područje označava uobičajeno vrijeme odgovora, a ne zajamčenu granicu; stvarno stanje usluge nije prikazano. Vrijeme teče udesno, a razmaci nisu u mjerilu.

5. **Zaštitite pravilo.** Dva pokušaja iste namjeravane predaje provjeravaju postoji li prihvaćeni zapis prije nego što ga ijedan stvori. Napišite ispreplitanje koje proizvodi dva

zapisa, navedite narušenu invarijantu i objasnite zašto dodavanje drugog poslužitelja samo po sebi ne rješava problem.

6. **Odaberite kompromis.** Usluga zadataka je nedostupna, ali postoje upute za zadatak u priručnoj pohrani. Raspravite kada je prikaz tih uputa prihvatljiv i zašto prihvaćanje nove predaje može tražiti jače jamstvo.

7. **Navedite model.** Za portal navedite jednu pretpostavku o mreži, čvorovima i vremenu. Imenujte pravilo koje treba očuvati i dodatni uvjet potreban za konačno prihvaćanje. Što se mijenja tijekom trajne particije?

8. **Domaća zadaća: računi 1, 2, 3, ...** Tvrtka izdaje račune s više poslužitelja, a brojevi računa moraju ići slijedno: 1, 2, 3, … bez ponavljanja i bez preskakanja. Navedite invarijantu. Dva poslužitelja izdaju račun u istom trenutku: tko dodjeljuje broj? Poslužitelj uzme broj 42 i sruši se prije spremanja računa: što je s brojem 42? Blagajni istekne vrijeme čekanja pa pokuša ponovno: kako izbjeći izdavanje dvaju računa? Na kraju, postoji li dizajn u kojemu ništa ne žrtvujete — ni dostupnost, ni brzinu, ni strogi globalni slijed? Ako ne postoji, odlučite što biste žrtvovali i obrazložite izbor.

12.1 Mogući smjer razmišljanja

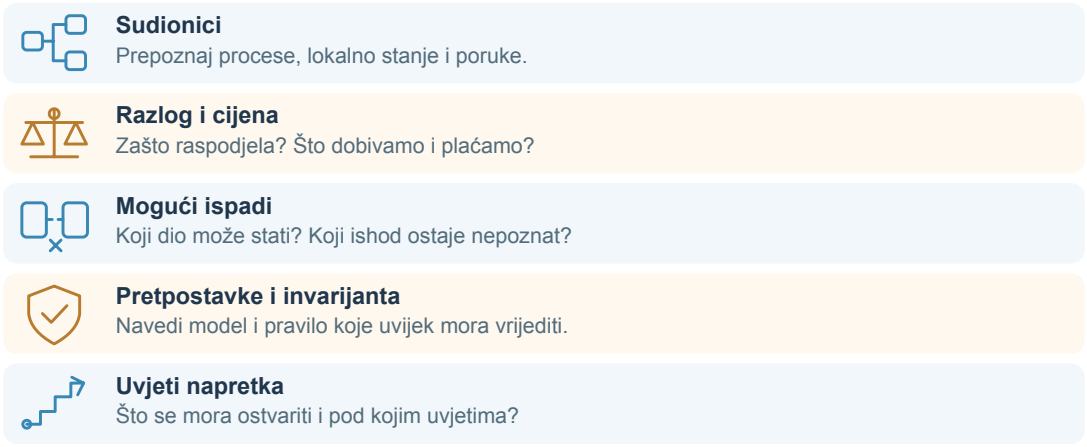
1. Klijent posjeduje svoj trenutačni prikaz i zahtjev na čekanju; usluga predaje stanje obrade; usluga zadataka rokove i pravila datoteka; pohrana predaja cijele datoteke projekata i zapise o prihvaćenim predajama. Zajedničko računalo stvara zajedničku domenu ispada. Točna podjela vlasništva u stvarnom dizajnu može se razlikovati i treba je navesti.
2. Zadaci se preklapaju u tijeku, pa je izvođenje konkurentno. Trag jednog radnog procesa ne pokazuje istodobno izvršavanje i stoga ne pokazuje paralelizam.
3. Zahtjev se mogao izgubiti, ili je mogao biti primijenjen uz izgubljen odgovor. Dulji konačni rok mijenja pravilo čekanja, ali ne daje granicu koju model ne jamči.
4. Opaženo tipično kašnjenje nije zajamčeni maksimum koji uključuje mrežu i obradu. „Nema odgovora do roka; sumnjamo na nedostupnost i pokrećemo definirani postupak oporavka” obranjiva je tvrdnja o opažanju.
5. A ne pronalazi zapis, B ne pronalazi zapis, A stvara prihvaćeni zapis, zatim B stvara još jedan. Invarijanta jednog zapisa po namjeravanoj predaji je narušena. Za ispravnost treba mehanizam koji usklađuje provjeru i učinak; samo umnožavanje neusklađene implementacije ostavlja problem.
6. Zastarjele upute za zadatak mogu biti prihvatljive ako su prikladno označene. Prihvaćanje predaje može ovisiti o trenutačnom roku te dopuštenim vrstama i veličinama datoteka. Odluka slijedi zahtjeve operacije i bilo koju izričito dopuštenu zastarjelost.
7. Jedan mogući model dopušta gubitak i promjenu redoslijeda, ispad s oporavkom uz trajne prihvaćene zapise i neograničena kašnjenja odgovora. Treba očuvati pravilo bez duplikata. Konačno prihvaćanje dodatno traži oporavak, konačnu komunikaciju, sačuvan ili ponovljen posao i obradu. Trajna particija ruši komunikacijski uvjet za napredak; ne ublažava sigurnost. Prihvatljivi su i drugi izričiti, dosljedni izbori.

8. Invarijanta ima dva dijela: nijedan broj nije upotrijebljen dvaput (sigurnost od duplikata) i nijedan broj nije preskočen (bez rupa). Jedna usluga brojača zadovoljava oba dijela samo dok je dostupna; postaje usko grlo i jedinstvena točka ispada. Rušenje nakon dodjele broja 42, a prije spremanja računa, stvara rupu, osim ako dodjela i spremanje nisu atomarni ili se broj 42 ne zabilježi kao storniran. Ponovni pokušaj nakon isteka čekanja stvara duplikat, osim ako zahtjev nosi identifikator, a poslužitelj za ponovljeni identifikator vrati postojeći račun (idempotentnost, kao u Web aplikacijama). Ako svaki poslužitelj dobije vlastiti raspon brojeva, brojevi ostaju jedinstveni i dostupni, ali se gubi globalni slijed. Zasebno brojenje po blagajni ili poslovnom prostoru čuva strogi slijed unutar svakoga od njih, a tako račune označavaju mnogi fiskalni sustavi. Nijedan dizajn ne žrtvuje ništa: globalni slijed bez rupa traži koordinaciju, pa tijekom particije ili rušenja koordinatora sustav mora ili čekati, gubeći dostupnost, ili nastaviti bez koordinacije, gubeći slijed. Replikacija (predavanje 06), konsenzus (predavanje 07) i transakcije (predavanje 09) razvijaju koordinirane mogućnosti.

13 Sažetak

13.1 Skrivene pretpostavke koje treba preispitati

Klasične zablude raspodijeljenog računarstva (fallacies of distributed computing) upozoravaju na prešutne pretpostavke: pouzdanu dostavu, nultu latenciju, neograničen kapacitet veze, ugrađenu sigurnost, nepromjenjivu topologiju, jednog administratora, besplatan prijenos podataka i jednake komponente. Praktičan odgovor je navesti stvarne uvjete i odlučiti kako postupiti kada ne vrijede. Na primjer, izgubljeni odgovor dovodi u pitanje pouzdanu dostavu, a spor odgovor ideju zanemarive latencije. Više timova može nametati različita operativna pravila; kompatibilna sučelja moraju se dizajnirati unatoč različitim verzijama i implementacijama. To ne zabranjuje jače apstrakcije: traži objašnjenje kako se ostvaruju i gdje njihova jamstva prestaju. Vidi Oracleov prikaz.



Slika 21: Popis za razmišljanje: prepoznajte sudionike i poruke, odvagnite koristi i troškove, razmotrite ispade, navedite pretpostavke i invarijante, zatim odredite uvjete napretka.

Raspodijeljene komponente surađuju komunikacijom, a zadržavaju neovisno izvršavanje i lokalno stanje. Raspodjela može ispuniti korisne zahtjeve, ali stvara posao koordinacije i oporavka. Konkurentnost ne traži paralelni hardver. Djelomični ispad može ostaviti neke procese da rade dok operacija postaje nedostupna. Najvažnije: istek vremena čekanja sam po sebi ne određuje ishod udaljene operacije.

Za svaki predloženi dizajn pitajte: **tko sudjeluje, koje poruke prelaze granice, što mora ostati istinito i koje pretpostavke opravdavaju predloženo ponašanje?**

13.2 Osmislite sljedeći korak: jesam li predala na vrijeme?

Rok je u **ponoć**. Prijenos studentice završi u **23:59**, ali provjere završe u **00:19**. **Kako biste dizajnirali portal da studentica zna je li predala na vrijeme — čak i nakon što zatvori preglednik?** Predložite što portal treba spremiti i prikazati, pravilo koje treba očuvati te svoje pretpostavke o mreži, ponovnom pokretanju i vremenu.

Najprije navedite pravilo o roku: koji se događaj računa kao predaja? To što preglednik javi da je prijenos dovršen samo po sebi ne dokazuje da je usluga trajno primila projekt. Razlikujte primanje projekta prije roka od odluke prolazi li projekt provjere.

Smjerovi za istraživanje: potvrda za spremljeni prijenos, vrijeme primitka koje bilježi poslužitelj, status „primljeno, provjere u tijeku” i identifikator kojim se studentica kasnije može vratiti istoj predaji. Objasnite što svaki dio vašeg prijedloga utvrđuje. Više dizajna može biti obranjivo; nemojte pretpostaviti da potvrda dokazuje da je projekt prošao provjeru.

Proširite dizajn: što ako se poslužitelj sruši tijekom provjera? Koje informacije moraju preživjeti, kako se obrada nastavlja i kako studentica može pokušati ponovno a da ne stvori

još jednu predaju? Ta pitanja vode prema kasnijim predavanjima o granicama usluga, trajnom stanju i sigurnim ponovnim pokušajima.

14 Izvori i dodatno čitanje

[1] Maarten van Steen i Andrew S. Tanenbaum. *Distributed Systems, 4th edition*. 2023. Neobavezno šire čitanje iz udžbenika. Autori nude besplatan personalizirani PDF putem obrasca s e-poštom i CAPTCHA-om; poveznica za preuzimanje stiže e-poštom. Izravno dostupne bilješke [3] pokrivaju obavezne uvodne teme.

[2] Jim Waldo, Geoff Wyant, Ann Wollrath i Sam Kendall. *A Note on Distributed Computing*. Sun Microsystems Laboratories, 1994. Javna stranica autora s priloženim radom. Čitajte zbog razlike između lokalnog i udaljenog računanja, uključujući latenciju, konkurentnost i djelomični ispad.

[3] Martin Kleppmann. *Distributed Systems*. University of Cambridge, 2021./22. Javne bilješke s predavanja objavljene pod licencom CC BY-SA. Pročitajte odjeljke 1.1–1.3 za osnove i RPC, 2.1 za neizvjesnost i 2.3 za modele čvorova, mreže i vremena. Odjeljak 5.1 neobavezno je nastavno čitanje o ponovnim pokušajima i dupliciranim učincima.

[4] Jeffrey Dean i Luiz André Barroso. *The Tail at Scale*. Communications of the ACM 56(2), 2013. Kako kašnjenje rijetkih sporih odgovora raste kada zahtjev čeka mnogo usluga.